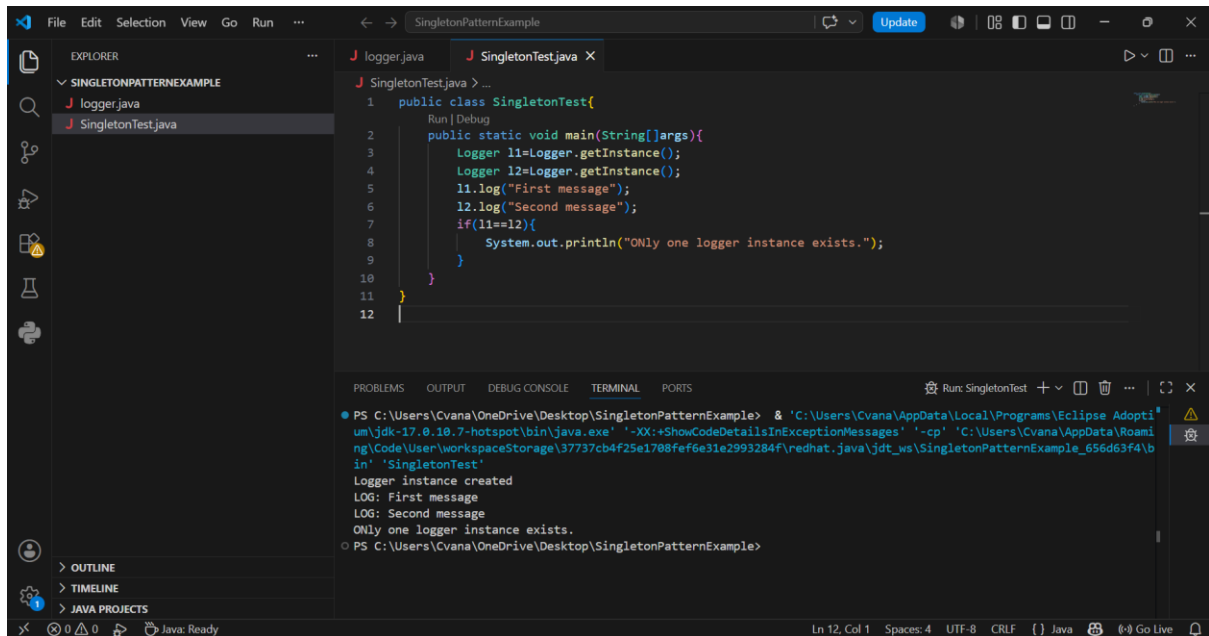


WEEK-1

Design Patterns And Principles

Exercise 1: Implementing the Singleton Pattern

Output:



The screenshot shows an IDE with a project named 'SingletonPatternExample'. The Explorer view on the left shows two files: 'logger.java' and 'SingletonTest.java'. The main editor displays the code for 'SingletonTest.java'.

```
1 public class SingletonTest{
2     Run | Debug
3     public static void main(String[] args){
4         Logger l1=Logger.getInstance();
5         Logger l2=Logger.getInstance();
6         l1.log("First message");
7         l2.log("Second message");
8         if(l1==l2){
9             System.out.println("Only one logger instance exists.");
10        }
11    }
12 }
```

The bottom panel shows the 'TERMINAL' view with the following output:

```
PS C:\Users\Cvana\OneDrive\Desktop\SingletonPatternExample> & 'C:\Users\Cvana\AppData\Local\Programs\Eclipse Adopti
um\jdk-17.0.10-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Cvana\AppData\Roami
ng\Code\User\workspaceStorage\37737cb4f25e1708fef6e31e2993284f\redhat.java\jdt_ws\SingletonPatternExample_656d63f4\b
in' 'SingletonTest'
Logger instance created
LOG: First message
LOG: Second message
Only one logger instance exists.
PS C:\Users\Cvana\OneDrive\Desktop\SingletonPatternExample>
```

Exercise 2: Implementing the Factory Method Pattern

Output:

```

Welcome | J Main.java X
J Main.java > WordDocument > open()
34 class ExcelFactory extends DocumentFactory {
36 }
37 public class Main {
    Run | Debug
38     public static void main(String[] args) {
39         DocumentFactory f1 = new WordFactory();
40         Document d1 = f1.createDocument();
41         d1.open();
42         DocumentFactory f2 = new PdfFactory();
43         Document d2 = f2.createDocument();
44         d2.open();
45         DocumentFactory f3 = new ExcelFactory();
46         Document d3 = f3.createDocument();
47         d3.open();
48     }
49 }

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS
• PS C:\Users\Cvana\OneDrive\Desktop\FactoryMethodPatternExample> javac Main.java
• PS C:\Users\Cvana\OneDrive\Desktop\FactoryMethodPatternExample> java Main
Opening Word Document
Opening PDF Document
Opening Excel Document
❖ PS C:\Users\Cvana\OneDrive\Desktop\FactoryMethodPatternExample>

```

Data Structures And Algorithms

Exercise 2: E-commerce Platform Search Function

Output:

```

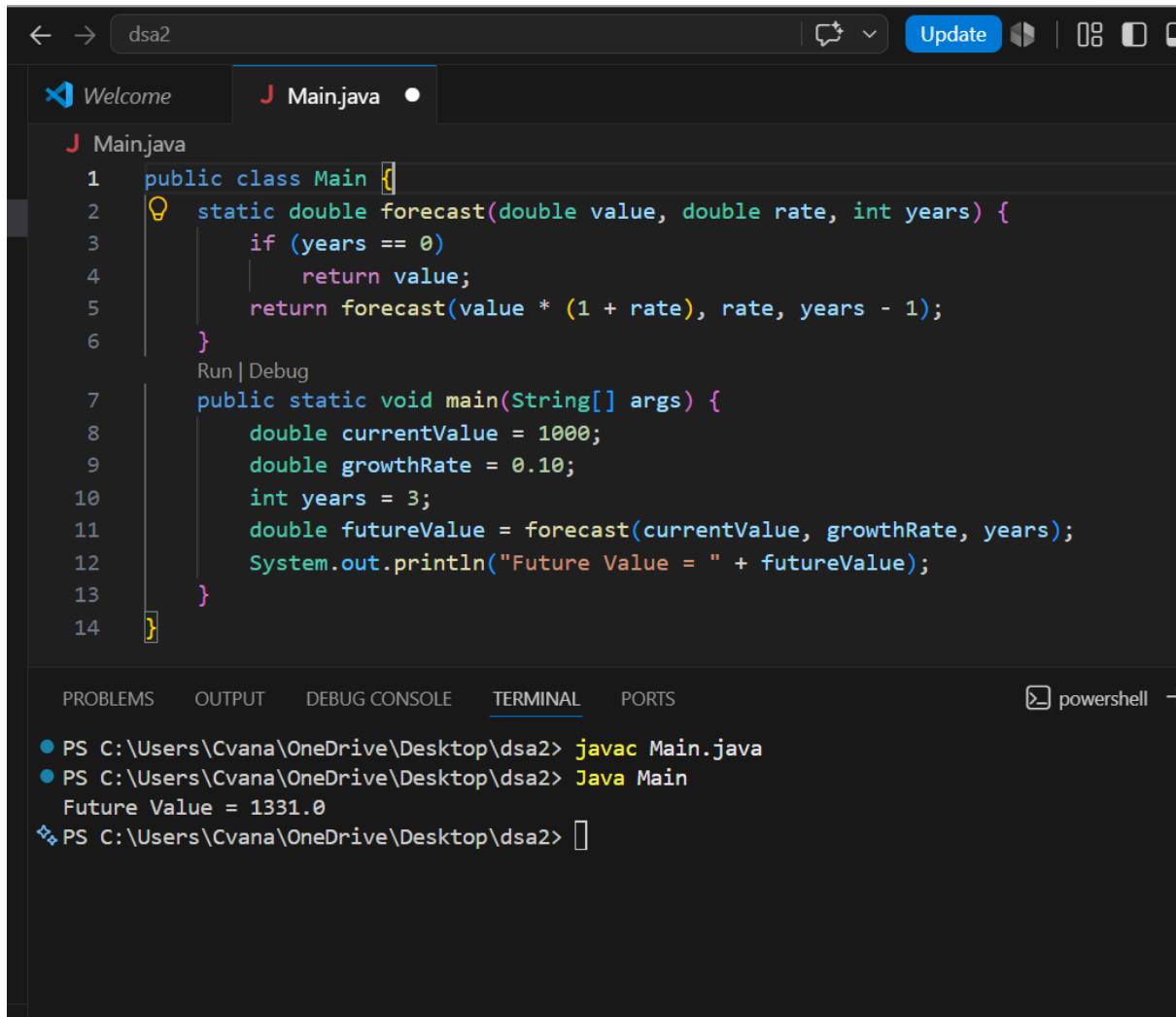
← → dsa1 | Update
Welcome | J Main.java X
J Main.java > Main > main(String[])
13 public class Main {
43     public static void main(String[] args) {
52
53         int searchId = 104;
54
55         Product p1 = linearSearch(products, searchId);
56
57         if (p1 != null)
58             System.out.println("Linear Search Found: " + p1.productName);
59
60         Product p2 = binarySearch(products, searchId);
61
62         if (p2 != null)
63             System.out.println("Binary Search Found: " + p2.productName);
64     }
65 }

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS
powershell ⚠ + ▾
• PS C:\Users\Cvana\OneDrive\Desktop\dsa1> javac Main.java
• PS C:\Users\Cvana\OneDrive\Desktop\dsa1> java Main
Linear Search Found: Watch
Binary Search Found: Watch
❖ PS C:\Users\Cvana\OneDrive\Desktop\dsa1>

```

Exercise 7: Financial Forecasting

Output:



The screenshot shows an IDE window titled 'dsa2' with a file named 'Main.java'. The code defines a recursive method 'forecast' and a 'main' method. The 'main' method calculates the future value of an investment starting at 1000 with a 10% growth rate over 3 years. The output of the program is displayed in the terminal window below the code editor.

```
1 public class Main {
2     static double forecast(double value, double rate, int years) {
3         if (years == 0)
4             return value;
5         return forecast(value * (1 + rate), rate, years - 1);
6     }
7     public static void main(String[] args) {
8         double currentValue = 1000;
9         double growthRate = 0.10;
10        int years = 3;
11        double futureValue = forecast(currentValue, growthRate, years);
12        System.out.println("Future Value = " + futureValue);
13    }
14 }
```

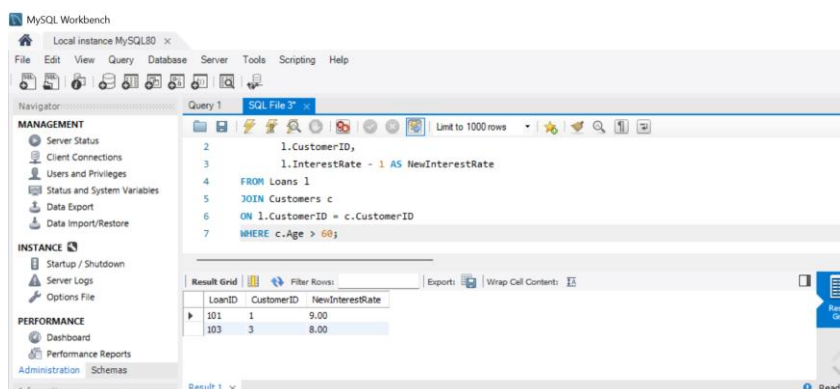
PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS powershell

- PS C:\Users\Cvana\OneDrive\Desktop\dsa2> javac Main.java
- PS C:\Users\Cvana\OneDrive\Desktop\dsa2> Java Main
- Future Value = 1331.0
- PS C:\Users\Cvana\OneDrive\Desktop\dsa2>

SQL

Exercise 1: Control Structures

Scenario1:



Scenario 2:

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports

Query 1 SQL File 3* x

```
1 • SELECT CustomerID,  
2       Balance,  
3       TRUE AS IsVIP  
4 FROM Customers  
5 WHERE Balance > 10000;
```

Result Grid

	CustomerID	Balance	IsVIP
1	1	15000.00	1
2	3	20000.00	1

Export: | Wrap Cell Content: |

Scenario 3:

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports

Administration Schemas

Query 1 SQL File 3* x

```
1 • SELECT c.CustomerID,  
2       l.LoanID,  
3       l.DueDate  
4 FROM Customers c  
5 JOIN Loans l  
6 ON c.CustomerID = l.CustomerID  
7 WHERE l.DueDate BETWEEN CURDATE()
```

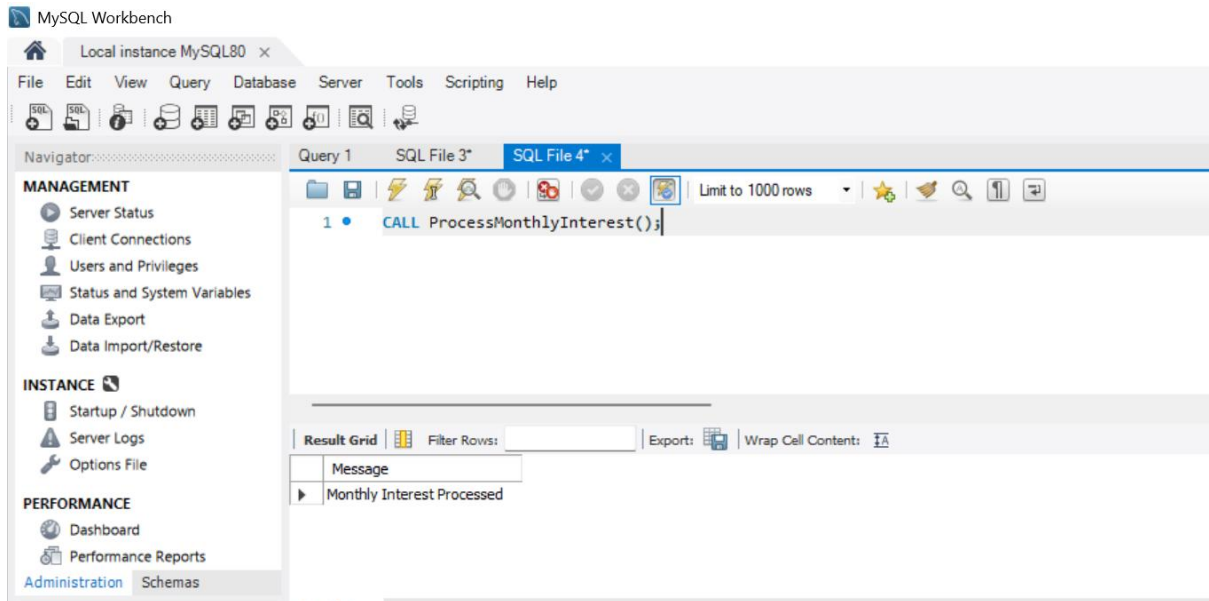
Result Grid

	CustomerID	LoanID	DueDate
1	1	101	2026-07-10
2	3	103	2026-07-15

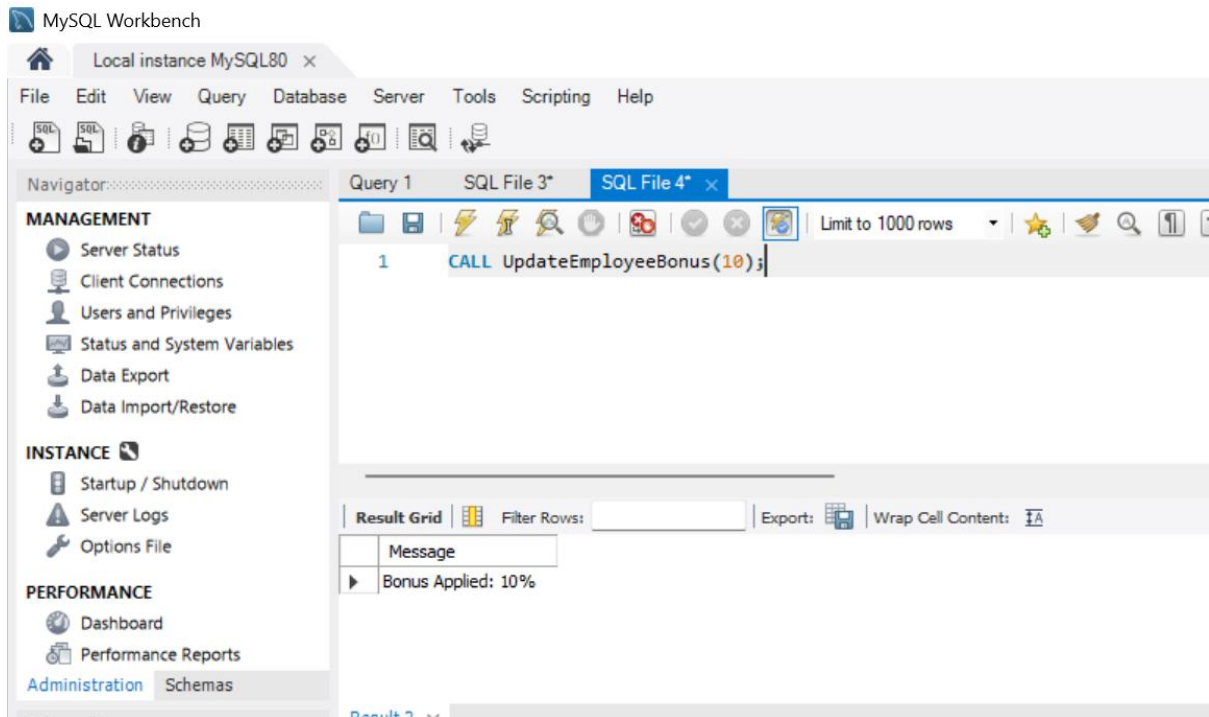
Export: | Wrap Cell Content: |

Exercise 3: Stored Procedures

Scenario 1:



Scenario 2:



Scenario 3:

